

Feature Selection in Statistical Machine Learning

Garvesh Raskutti

Lecture 6

Homework 1 and AI use/suggestions

- Homework 1 due Thursday week (Feb. 19)
- As mentioned earlier AI use allowed
- Try to use AI in the most beneficial way
- Remember the big assessments are project (AI use allowed) and exam (AI use NOT allowed)
- Use homework as a way to prepare for both
- Consider two modes: (1) unplugged from AI (useful for exam prep) and (2) plugged in to AI (useful for project)

Recommendations for unplugged from AI mode (useful for exam)

- For example in HW 1, Q1 (a)-(c) and all of Q2 useful for exam prep
- First try to do it yourself with no aids
- Second, either check or clear up confusion by using lecture slides/notes
- Third, use AI if you are really stuck but treat it as an absolute last resort
- After using AI, try to do the problem yourself without aid

Recommendations for plugged-in AI mode (useful for project)

- For example in HW 1, Q1 (d)-(e) and all of Q3 useful for project prep
- When using AI always adopt a *critical thinking* mindset
- Don't use AI as a teacher or expert, treat it more like a *learning assistant* where you are the expert
- Always thoroughly check AI outputs (often wrong or confused)
- Check everything you've written/done without AI help at the end

Recap and outline

- Last week we covered filter methods (correlation screening, mutual information screening, conditional correlation)
- Embedded methods
- Linear regression, Lasso, Elastic net
- Examples (2 simple synthetic models and 1 real data example)

- Example: Lasso/regularized loss
- Feature selection is part of prediction function estimation
- Parameters θ define feature selection $\{\widehat{\theta}_j = 0 \iff X_j \notin \widehat{S}\}$
- Only applies to function classes $\mathcal{F}$ that has such parameters (e.g. regression, decision trees, *not* neural networks)

Example: Linear regression

Linear regression objective:

$$\widehat{\beta} \in \arg \min_{\beta} \frac{1}{n} \sum_{i=1}^n (Y^{(i)} - \sum_{j=1}^p X_j^{(i)} \beta_j)^2 = \arg \min_{\beta} \frac{1}{n} \|Y - X\beta\|_2^2$$

where $Y \in \mathbb{R}^n$, $X \in \mathbb{R}^{n \times p}$ and $\beta \in \mathbb{R}^p$ Satisfies normal equations:

$$X^T X \widehat{\beta} = X^T Y$$

and if $X^T X$ is invertible

$$\widehat{\beta} = (X^T X)^{-1} X^T Y$$

Linear regression for bike-share data - code

- When p is small, the `sm.OLS` (OLS stands for ordinary least-squares) is very useful

```
import statsmodels.api as sm

X_ols = sm.add_constant(X)
model = sm.OLS(y, X_ols).fit()

print(model.summary())
```


Linear regression for bike-share data - results

OLS Regression Results						
=====						
Dep. Variable:	cnt	R-squared:	0.800			
Model:	OLS	Adj. R-squared:	0.797			
Method:	Least Squares	F-statistic:	261.9			
Date:	Tue, 03 Feb 2026	Prob (F-statistic):	7.80e-243			
Time:	10:00:06	Log-Likelihood:	-5981.0			
No. Observations:	731	AIC:	1.199e+04			
Df Residuals:	719	BIC:	1.204e+04			
Df Model:	11					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	1469.0031	240.218	6.115	0.000	997.390	1940.616
temp	2028.9161	1403.671	1.445	0.149	-726.867	4784.699
atemp	3573.2743	1589.389	2.248	0.025	452.877	6693.671
hum	-1018.8616	313.995	-3.245	0.001	-1635.318	-402.405
windspeed	-2557.5691	456.278	-5.605	0.000	-3453.365	-1661.774
season	509.7752	54.757	9.310	0.000	402.272	617.278
yr	2040.7034	65.185	31.306	0.000	1912.727	2168.680
mnth	-38.9796	17.079	-2.282	0.023	-72.510	-5.449
holiday	-518.9919	201.040	-2.582	0.010	-913.688	-124.296
weekday	69.0622	16.299	4.237	0.000	37.063	101.061
workingday	120.3570	72.007	1.671	0.095	-21.013	261.727
weathersit	-610.9870	78.363	-7.797	0.000	-764.835	-457.139
=====						
Omnibus:	88.811	Durbin-Watson:	0.967			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	180.069			
Skew:	-0.717	Prob(JB):	7.92e-40			
Kurtosis:	4.964	Cond. No.	561.			
=====						

How do we get p-values?

If $X^T X \in \mathbb{R}^{p \times p}$ is invertible:

$$\widehat{\beta} = (X^T X)^{-1} X^T Y$$

In general if $\widehat{\beta}_j$ is small, we don't select X_j and vice-versa
Linear model:

$$Y = X\beta + w$$

where $w \sim \text{Normal}(0, \sigma^2 I_{n \times n})$. Under a normal linear model:

$$\widehat{\beta} = (X^T X)^{-1} X^T (X\beta + w) \sim \text{Normal}(\beta, \sigma^2 (X^T X)^{-1})$$

Null hypothesis and p-values

Under a normal linear model;

$$\widehat{\beta} \sim \text{Normal}(\beta, \sigma^2(X^T X)^{-1})$$

For each feature X_j

$$\mathcal{H}_0 : \beta_j = 0 \quad \mathcal{H}_A : \beta_j \neq 0$$

Distribution under null hypothesis:

$$\widehat{\beta}_j \sim \text{Normal}(0, \sigma^2[(X^T X)^{-1}]_{jj})$$

Estimate σ^2 by using residuals formula:

$$S^2 = \frac{1}{n - p - 1} \sum_{i=1}^n (Y^{(i)} - \sum_{j=1}^p \widehat{\beta}_j X_j^{(i)})^2$$

Then use t-statistic/t-test:

$$t_j = \frac{\widehat{\beta}_j}{S \sqrt{[(X^T X)^{-1}]_{jj}}} \sim t_{n-p-1}$$

under $\mathcal{H}_0$

Comparison to conditional correlation screening

For linear regression, t-statistic is:

$$t_j^{\text{reg}} = \frac{\widehat{\beta}_j}{S \sqrt{[(X^T X)^{-1}]_{jj}}} \sim t_{n-p-1}$$

For conditional correlation, t-statistic is:

$$t_j^{\text{pcor}} = \widehat{\rho}_{j|-j} \sqrt{(n-p-1)/(1-\widehat{\rho}_{j|-j}^2)}$$

Using linear algebra (Frisch-Waugh-Lovell theorem), you can prove that

$$t_j^{\text{reg}} = t_j^{\text{pcor}} \sim t_{n-p-1}$$

Thus partial correlation and linear regression are equivalent up to t-statistic and p-value

Limitations of linear regression

- Tends to only capture linear relationships
- Relies on $X^T X$ being invertible (problematic when p is large and/or features are very highly correlated)

Beyond linear regression: Generalized linear models

Linear regression:

$$\widehat{\beta} \in \arg \min_{\beta} \frac{1}{n} \sum_{i=1}^n (Y^{(i)} - \sum_{j=1}^p X_j^{(i)} \beta_j)^2$$

- If data is binary/count e.t.c. often does not make sense to use least-squared loss
- A natural and clear extension is *generalized linear models* where we replace least-squares loss with convex differentiable loss function

$$\widehat{\beta} \in \arg \min_{\beta} \frac{1}{n} \sum_{i=1}^n \ell(Y^{(i)}, \sum_{j=1}^p X_j^{(i)} \beta_j)$$

where $\ell(., .)$ is a *convex* loss function.

Convex loss functions

$$\widehat{\beta} \in \arg \min_{\beta} \frac{1}{n} \sum_{i=1}^n \ell(Y^{(i)}, \sum_{j=1}^p X_j^{(i)} \beta_j)$$

- Include logistic regression, Poisson regression and support vector machines (SVMs) as special cases (as well as others)
- Under suitable generative model

$$\beta_j = 0 \Leftrightarrow Y \not\perp\!\!\!\perp X_j | X_{-j}$$

What happens when $X^T X$ not invertible?

- Very common in practice
- Redundancy/high dependence in groups of features (e.g. $X_3 = X_1 + X_2$ and $Y = 3X_1 + 2X_2$)
- What if $p > n$ (then $X^T X$ can't be invertible)

Non-identifiability of feature selection

- If $X^T X$ not invertible feature selection is *non-identifiable*
- Meaning there are many ways to choose the same feature set that give the same solution
- With highly correlated features S^* is not unique
- In the high-dimensional setting, n is not sufficiently large to determine a solution

Highly correlated features example

Consider a linear model where:

$$Y = X_1 + \frac{1}{2}X_2$$

but the features are perfectly correlated:

$$X_2 = 2X_1$$

If you fit a linear model

$$Y = \beta_1 X_1 + \beta_2 X_2,$$

any solution (β_1, β_2) of the form

$$\beta_1 + 2\beta_2 = 2$$

works.

High-dimensional $p > n$ example

Need to fit a linear model where $p > n$:

$$Y = \sum_{j=1}^p \beta_j X_j.$$

For our data assume $Y^{(i)} = r_j$ and

$$\begin{aligned} X_j^{(i)} &= 1 \text{ if } i = j \\ X_j^{(i)} &= 0 \text{ if } i \neq j \end{aligned}$$

Each new sample selects a different mutation.

Then matrix $X^T X$ is a diagonal matrix with n diagonal entries 1 and the other $p - n$ entries are 0.

Example continued

Linear regression satisfies normal equations:

$$X^T X \widehat{\beta} = X^T Y$$

For $1 \leq j \leq n$, we get the equations:

$$\widehat{\beta}_j = r_j$$

while for $n + 1 \leq j \leq p$ we get the equations:

$$0 \cdot \widehat{\beta}_j = 0$$

which has infinite solutions.

How do we address this non-identifiability?

Issue is non-uniqueness

$$\widehat{\beta}^{\text{OLS}} \in \arg \min_{\beta} \sum_{i=1}^n (Y^{(i)} - \sum_{j=1}^p X_j^{(i)})^2$$

Add regularization to make solution (possibly) unique:

$$\widehat{\beta}^{\lambda, \mathcal{R}} \in \arg \min_{\beta} \sum_{i=1}^n (Y^{(i)} - \sum_{j=1}^p X_j^{(i)})^2 + \lambda \mathcal{R}(\beta)$$

where $\mathcal{R}(\cdot)$ is regularization/penalization function and $\lambda > 0$ is scalar regularization parameter

Examples

$$\widehat{\beta}^{\lambda, \mathcal{R}} \in \arg \min_{\beta} \sum_{i=1}^n (Y^{(i)} - \sum_{j=1}^p X_j^{(i)})^2 + \lambda \mathcal{R}(\beta)$$

Ridge regression: $\mathcal{R}(\beta) = \sum_{j=1}^p \beta_j^2 = \|\beta\|_2^2$

AIC or BIC (different choices of λ): $\mathcal{R}(\beta) = \sum_{j=1}^p 1(\beta_j \neq 0) := \|\beta\|_0$

Lasso : $\mathcal{R}(\beta) = \sum_{j=1}^p |\beta_j| = \|\beta\|_1$

Elastic net: $\mathcal{R}(\beta) = \lambda_1 \|\beta\|_1 + \lambda_2 \|\beta\|_2^2$

- Ridge regression always guarantees a unique solution if $\lambda > 0$
- AIC and BIC perfect for feature selection but non-computable/NP-hard
- Lasso ideal compromise as it performs feature selection but leads to a convex optimization problem
- Elastic net is balance between Lasso (good for feature selection) and ridge (good for prediction)

Real data example

- Breast cancer $n = 570$, $p = 180$
- Highly correlated group-structured features
- We apply OLS, Lasso and elastic net

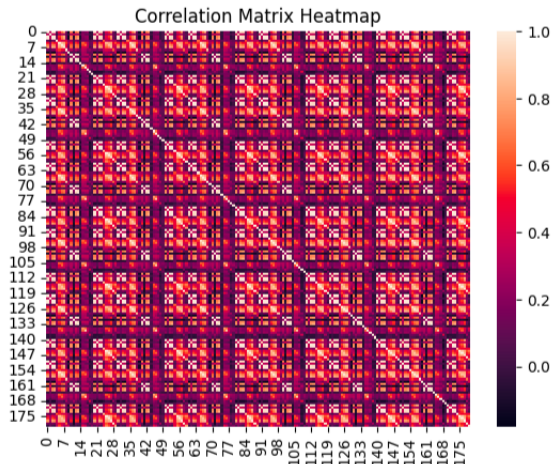
Correlation heat map for features -code

```
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Compute the correlation matrix
df_X = pd.DataFrame(X)

# 1. Calculate the correlation matrix from the feature matrix
corr_matrix = df_X.corr()
|
# 2. Plot the heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix)
plt.title('Correlation Matrix Heatmap')
plt.show()
```


Correlation heat map for features -heatmap



- Note: many highly correlated features
- Clusters of features that are highly correlated

OLS vs Lasso vs Elastic net -code

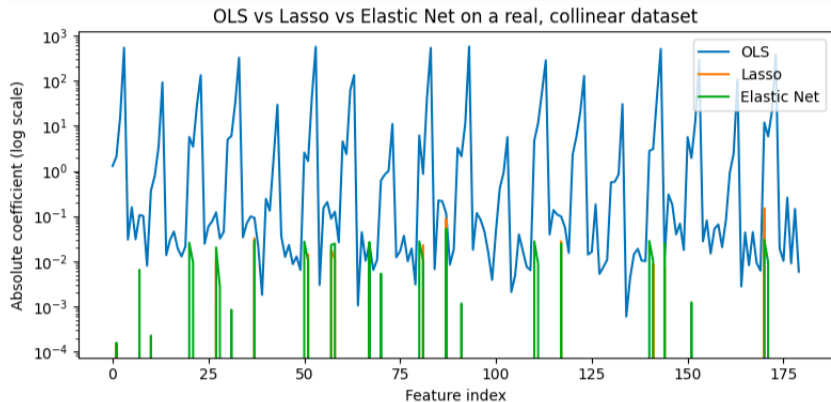
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Lasso, ElasticNet

# -----
# 1. Standardize features (important!)
# -----
X_hd = StandardScaler().fit_transform(X)

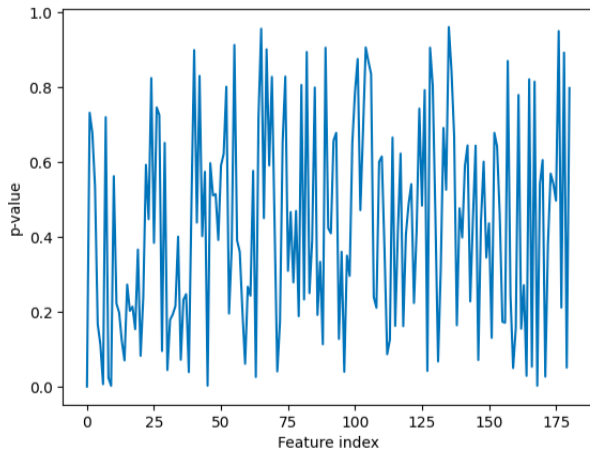
# -----
# 2. Fit models
# -----
ols = LinearRegression().fit(X_hd, y)
lasso = Lasso(alpha=0.05, max_iter=10000).fit(X_hd, y)
enet = ElasticNet(alpha=0.05, l1_ratio=0.5, max_iter=10000).fit(X_hd, y)

# -----
# 3. Plot coefficient magnitudes
# -----
plt.figure(figsize=(8, 4))
plt.plot(np.abs(ols.coef_), label="OLS")
plt.plot(np.abs(lasso.coef_), label="Lasso")
plt.plot(np.abs(enet.coef_), label="Elastic Net")
plt.yscale("log")
plt.xlabel("Feature index")
plt.ylabel("Absolute coefficient (log scale)")
plt.title("OLS vs Lasso vs Elastic Net on a real, collinear dataset")
plt.legend()
plt.tight_layout()
plt.show()
```

OLS vs Lasso vs Elastic net -code

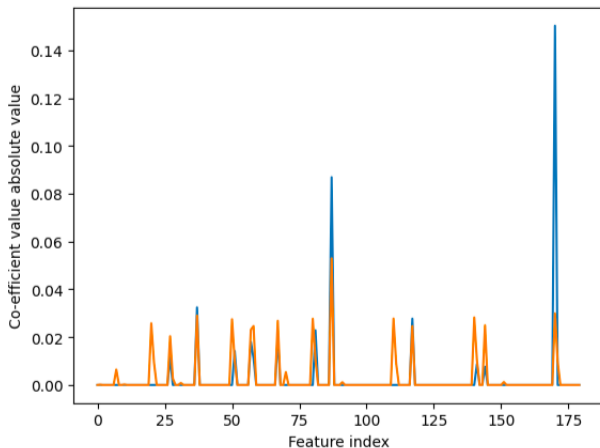


OLS p-values



- Notice that there is basically no feature reduction

Lasso and Elastic Net co-efficients



- Note many features removed (very sparse)

- Linear regression (unregularized) tends to select most or all features
- Lasso and Elastic Net tend to pick few features (encourage sparsity)
- If features are highly correlated, Lasso will usually just pick a single feature
- The ℓ_1 norm automatically encodes feature selection

Returning to highly correlated example

Consider a linear model where:

$$Y = X_1 + \frac{1}{2}X_2$$

but the features are perfectly correlated:

$$X_2 = 2X_1$$

If you fit a linear model

$$Y = \beta_1 X_1 + \beta_2 X_2,$$

any solution of the form

$$\beta_1 + 2\beta_2 = 2$$

works.

What does Lasso do?

- Minimize $\|\beta\|_1$ subject to $\beta_1 + 2\beta_2 = 2$
- Minimize $|\beta_1| + |\beta_2|$ subject to $\beta_1 + 2\beta_2 = 2$
- Minimized at $(\beta_1, \beta_2) = (0, 1)$
- Hence a sparse solution

What do OLS/ridge regression do?

- OLS can not be computed due to lack of identifiability
- In the limit OLS behaves like ridge regression where we minimize $\|\beta\|_2^2$ subject to $\beta_1 + 2\beta_2 = 2$
- Minimize $\beta_1^2 + \beta_2^2$ subject to $\beta_1 + 2\beta_2 = 2$
- Minimized at $(\beta_1, \beta_2) = (\frac{2}{5}, \frac{4}{5})$
- Note solution is not sparse
- This explains example where OLS provides a totally non-sparse solution whereas Lasso is sparse

Summary

- Regularized/penalized regression is a very standard embedded method
- Different regularizers induce different structures
- For feature selection ℓ_1 -penalization very common and widely used
- ℓ_1 -penalization creates sparse solution as does feature selection automatically
- Lasso resolves highly dependent features by selecting one
- OLS/ridge usually splits the weight among features so non-sparse solutions